

D. Merge, Rebase, Squash!

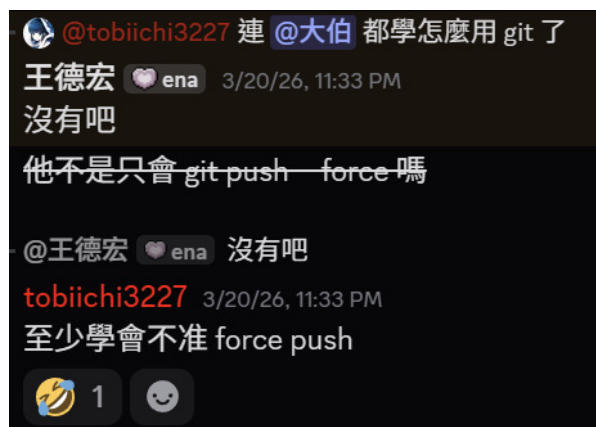
Problem ID: Git

Time Limit: 3.5s

Memory Limit: 512MiB



Force push 1



Force push 2

Figure 1: 不要學大伯 force push (他現在已經不會了)

tobiichi3227 是 TOJ 的維護者。TOJ 的程式碼由 H 個互相獨立的 hunk 組成，正式版本位於 branch v2.0。

王德宏身為 TOJ 開發團隊的成員之一，有天想為 TOJ 貢獻一個酷炫的功能，於是很高興地開啟了新的 branch feat/fancy，並在上面依序進行了 N 次操作。

王德宏想讓這項功能進到 TOJ 的 branch v2.0，於是請 tobiichi3227 進行審核。tobiichi3227 覺得 N 個 commits 太多了，應該先用 squash 將它們整理成恰好 K 個 squash commits，再依序 rebase 到 v2.0。

不同的 squash 方式可能產生不同的 conflict 成本。然而，王德宏要去參加國際地理奧林匹亞 (IGGO，全名 International GeoGuesser Olympiad)，已經沒有時間處理了，於是把最小化 conflict 成本的任務交給正在看題目的你。

不用擔心，下面會告訴您 branch rebase squash conflict 的定義。

完整流程合併動畫：[點我下載完整合併 GIF](#)。

– Branch 與狀態 –

feat/fancy 一開始的所有 hunk 內容皆為 0。令 $S_i[h]$ 表示執行完前 i 次操作後，第 h 個 hunk 的內容，因此：

$$S_0[h] = 0.$$

第 i 次操作會把第 x_i 個 hunk 改成 v_i ，也就是：

$$S_i[h] = \begin{cases} v_i, & h = x_i, \\ S_{i-1}[h], & h \neq x_i. \end{cases}$$

另一方面，v2.0 上第 h 個 hunk 的內容為 U_h 。



Figure 2: 兩個 branches 從共同位置分開前進

完整動畫：[點我下載 Branch GIF](#)。

– Squash –

你需要將這 N 次操作依序切分成恰好 K 個連續的區間（即 K 個 squash commits）。

假設其中一個區間包含了第 L 到第 R 次操作（ $1 \leq L \leq R \leq N$ ），該區間對第 h 個 hunk 的實質修改定義為：

- 修改前（舊內容）： $\text{old} = S_{L-1}[h]$
- 修改後（新內容）： $\text{new} = S_R[h]$

若 $\text{old} = \text{new}$ ，則視為該區間對第 h 個 hunk 沒有進行修改。

等價地，可以選擇：

$$0 = p_0 < p_1 < \cdots < p_K = N,$$

其中第 g 個區間為 $[p_{g-1} + 1, p_g]$ 。

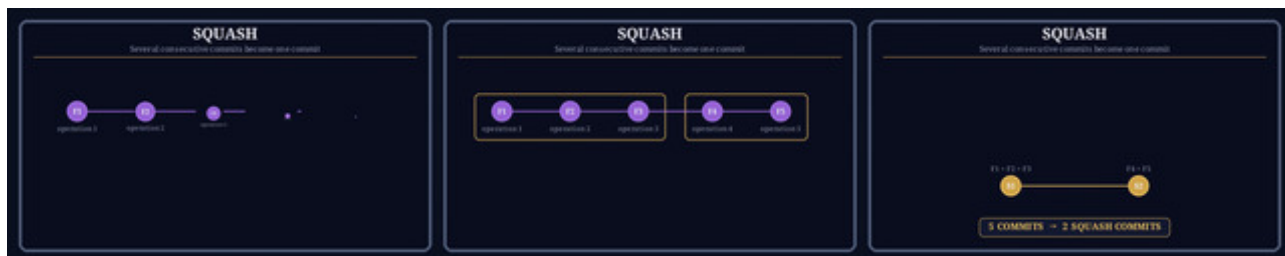


Figure 3: 每個連續區間形成一個 squash commit

完整動畫：[點我下載 Squash GIF](#)。

– Rebase –

這 K 個區間會按照原本順序逐一套用。令 $\text{current}[h]$ 表示 v2.0 上第 h 個 hunk 目前的內容；一開始：

$$\text{current}[h] = U_h.$$

處理一個區間時，會分別考慮每個 hunk 的 old、new 與 $\text{current}[h]$ 。



Figure 4: 各 squash commits 依序 replay 到 v2.0

完整動畫：[點我下載 Rebase GIF](#)。

– Conflict –

若 $old = new$ ，該 hunk 沒有被區間修改， $current[h]$ 不變。

否則，當且僅當：

$$current[h] \neq old \quad \text{且} \quad current[h] \neq new$$

時會發生 conflict，並產生 w_h 的成本。

不論是否發生 conflict，處理完這項實質修改後皆令：

$$current[h] \leftarrow new.$$

每個區間、每個 hunk 的 conflict 成本分別計算；你的目標是選擇 K 個區間，使總成本最小。



Figure 5: 發生 conflict 的兩個 commits 閃紅，解決後轉為綠色

完整動畫：[點我下載 Conflict GIF](#)。

— 輸入 —

第一行包含三個整數 N, H, K ，分別表示：

- feat/fancy branch 上的 commit 數量；
- hunk 的數量；
- 需要切分出的連續區間數量。

第二行包含 H 個整數 U_h ($1 \leq h \leq H$)

其中 U_h 表示 v2.0 branch 上第 h 個 hunk 的最終內容。

第三行包含 H 個整數 w_h ($1 \leq h \leq H$)

其中 w_h 表示解決第 h 個 hunk 的 conflict 所需的成本。

接下來 N 行，第 i ($1 \leq i \leq N$) 行包含兩個整數 x_i 與 v_i
表示第 i 個 commit 將第 x_i 個 hunk 的內容改成 v_i 。

— 輸出 —

輸出一個整數，表示最小的 conflict 解決總成本。

— 輸入限制 —

- $1 \leq N, H \leq 200000$
- $1 \leq K \leq \min(N, 30)$
- $1 \leq U_h \leq 10^9$
- $1 \leq w_h \leq 10^9$
- $1 \leq x_i \leq H$
- $1 \leq v_i \leq 10^9$
- 對於每個 commit $i : v_i \neq S_{i-1}[x_i]$
- 保證每個 commit 都會實際改變該 hunk 的內容，且 feat/fancy branch 不會將任何 hunk 改回 0。

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	3	$N, H \leq 15$
3	7	$K = 1$
4	9	$K = N$
5	19	每個 hunk 最多只被修改一次
6	13	$N \leq 1000$
7	49	無限制

— 範例輸入 —

```
5 2 2
1 4
5 3
1 1
2 2
1 2
2 4
1 3
```

— 範例輸出 —

```
0
```

— 範例說明 —

feat/fancy branch 上兩個 hunk 的狀態依序如下：

```
commit 0: (0, 0)
commit 1: (1, 0)
commit 2: (1, 2)
commit 3: (2, 2)
commit 4: (2, 4)
commit 5: (3, 4)
```

可以將 commits 分成：

[1] [2, 3, 4, 5]

第一個 squash commit 將 hunk 1 :

0 -> 1

套用到 v2.0 branch 時，hunk 1 的目前內容已經是 1，因此不會發生 conflict。

第二個 squash commit 的效果為：

hunk 1 : 1 -> 3

hunk 2 : 0 -> 4

此時 hunk 1 的目前內容為 1，可以正常套用。

hunk 2 的目前內容已經是 4，因此也不會發生 conflict。

總成本為 0。

— 範例輸入 —

```
2 1 1
1
7
1 1
1 2
```

— 範例輸出 —

```
7
```

— 範例說明 —

因為 $K = 1$ ，兩個 commits 必須全部 squash 在一起。
Squash 後的效果為：

```
0 -> 2
```

但 v2.0 branch 上該 hunk 的內容為 1。

目前內容既不是 patch 的舊內容 0，也不是新內容 2，因此發生 conflict，成本為 7。

— 範例輸入 —

```
2 1 2
1
7
1 1
1 2
```

— 範例輸出 —

```
0
```

— 範例說明 —

因為 $K = 2$ ，兩個 commits 分別套用。
第一個 commit 的效果為：

```
0 -> 1
```

而 v2.0 branch 上目前的內容已經是 1，因此不會發生 conflict。
第二個 commit 的效果為：

1 -> 2

此時目前內容為 1，可以正常套用。
因此總成本為 0。

點我跳過 Git 小教室

– Git 小教室 –

– 下面內容不閱讀不影響解題 –

這裡用一個小型 repository，把題目裡的 branch、squash、rebase 和 conflict 真的實現一次。

不需要網路，toj.git 是放在同一台電腦上的本機 remote，不會連到 GitHub。

以下命令以 Bash 或 Git Bash 為例，假設大家會基本的 Linux CLI。

Git 到底在做什麼？

先解釋後面會出的名詞

- **working tree**：現在資料夾裡看到的檔案。
- **staging area**：下一個 commit 要收進去的內容。
- **commit**：某個時間點的檔案狀態，以及它的前一個 commit。
- **branch**：指向某個 commit 的名稱。建立新 commit 後，branch 會跟著往前走，例如 v2.0、feat/fancy。
- **HEAD**：你現在所在的位置，通常是某個 branch。
- **remote**：另一個 repository 的名稱。它可以在網路上，也可以只是本機的一個路徑。

平常用到的命令大概是這些：

<code>git init</code>	# 建立 <i>repository</i>
<code>git status</code>	# 查看目前 <i>branch</i> 、已修改與已 <i>stage</i> 的檔案
<code>git diff</code>	# 查看尚未 <i>stage</i> 的差異
<code>git diff --staged</code>	# 查看已 <i>stage</i> 、將進入下一個 <i>commit</i> 的差異
<code>git add <file></code>	# 將檔案的目前內容放進 staging area
<code>git commit -m "<message>"</code>	# 建立 <i>commit</i>
<code>git log --graph --oneline --all</code>	# 用簡圖查看所有 <i>branch</i> 的歷史
<code>git branch</code>	# 列出 <i>branch</i>
<code>git switch <branch></code>	# 切換 <i>branch</i>
<code>git switch -c <branch></code>	# 建立並切換到新 <i>branch</i>
<code>git merge <branch></code>	# 把指定 <i>branch</i> 合併到目前 <i>branch</i>
<code>git rebase <branch></code>	# 把目前 <i>branch</i> 的 <i>commits</i> 逐一重播到指定 <i>branch</i>

`git commit -am "<message>"` 可以看成 `git add` 已追蹤檔案再 `git commit` 的縮寫。
不過，新建立跟還沒被 Git 追蹤的檔案不會被加進去。

不確定現在 repository 是什麼狀態時，先執行：`git status`

1. 建立完全離線的 repository

先建立一個新的資料夾。toj.git 當作遠端的 TOJ repository，work 則是平常工作的 repository：

```
mkdir git-classroom
cd git-classroom
git init --bare toj.git
git init -b v2.0 work
cd work
```

bare repository 沒有 working tree，所以適合拿來當 remote。

接著只設定這次教學使用的作者資料，email 隨便填就好。

```
git config user.name "Git Student"
git config user.email "student@example.com"
```

接下來用兩個一行文字檔模擬兩個 hunk，先建立共同的起點：

```
printf "0\n" > hunk-1.txt
printf "0\n" > hunk-2.txt
git add hunk-1.txt hunk-2.txt
git status
git commit -m "base: all hunks are 0"
git tag base
```

base tag 只是幫這個 commit 取一個固定名字。後面做 squash 時，可以直接寫 base，不需要一直複製 commit hash。

2. 在 feat/fancy 建立四個 commits

從共同的起點開一個 feature branch 並切換過去：

```
git switch -c feat/fancy
```

接著改幾次檔案，每次都做一次 commit。這裡故意讓同一個 hunk 被修改不只一次：

```
printf "3\n" > hunk-1.txt
git commit -am "fancy 1: hunk 1, 0 to 3"

printf "4\n" > hunk-2.txt
git commit -am "fancy 2: hunk 2, 0 to 4"

printf "5\n" > hunk-1.txt
git commit -am "fancy 3: hunk 1, 3 to 5"

printf "7\n" > hunk-1.txt
git commit -am "fancy 4: hunk 1, 5 to 7"
```

現在可以 git log 看這四個 feature commits：

```
git log --oneline --decorate
```

把本機的 bare repository 設成 origin，再把目前的 branch 推上去：

```
git remote add origin ../toj.git
git push -u origin feat/fancy
```

-u 會記住本機 feat/fancy 對應的是 origin/feat/fancy。

這裡的 remote 是 ../toj.git，所以整個 push 都是在本機操作。

3. 讓 v2.0 也往前走

切回 v2.0，把兩個 hunk 設成最後的 (5, 4)：

```
git switch v2.0
printf "5\n" > hunk-1.txt
printf "4\n" > hunk-2.txt
git commit -am "v2.0: set final hunk states"
git push -u origin v2.0
git log --graph --oneline --decorate --all
```

現在兩條 branch 從 base 分開了：

```
          fancy 1 -- fancy 2 -- fancy 3 -- fancy 4  (feat/fancy)
        /
base: (0, 0) --
        \
          v2.0: (5, 4)                                (v2.0)
```

4. Merge 和 Rebase 差在哪裡？

先看 merge。

這裡另外開一條 merge-demo，不影響後面真正要做的 rebase：

```
git switch -c merge-demo v2.0
git merge feat/fancy
```

這時 Git 會遇到 conflict。

原因很簡單，因為兩邊都改了 hunk-1.txt，但改成的內容不同。

hunk-2.txt 就沒有這個問題。兩邊最後都是 4，所以 Git 可以自己處理。

可以用下面的命令看目前狀態：

```
git status
git diff
```

這次只是拿 merge 來比較，所以不需要留下這個結果。放棄 merge，回到 feature branch：

```
git merge --abort
git switch feat/fancy
```

rebase 的做法不一樣。

執行：

```
git rebase v2.0
```

Git 會先找到 feat/fancy 和 v2.0 的共同祖先 (LCA)，接著把 feature branch 上的 commits 暫時拿下來，再一個一個套到新版的 v2.0 後面，所以不會出現像 cerge 一樣的 merge commit。

有一點要注意：rebase 不是直接把 branch 名字搬過去，commit 重新建立後，parent 和內容雜湊可能都會改，所以 commit hash 也會跟著變。

5. 把四個 commits squash 成兩個

題目要求把連續的 commits 分組。

這裡分成 [1,2] [3,4]，所以最後會剩兩個 commits，也就是 $K = 2$ 。

目前在 feat/fancy 上，執行：

```
git rebase -i base
```

編輯器會列出四個 commits，順序是由舊到新。hash 每台電腦都可能不同，所以只看排列就好：

```
pick <hash 1> fancy 1: hunk 1, 0 to 3
pick <hash 2> fancy 2: hunk 2, 0 to 4
pick <hash 3> fancy 3: hunk 1, 3 to 5
pick <hash 4> fancy 4: hunk 1, 5 to 7
```

每一組裡，第一個維持 pick，後面的改成 squash：

```
pick    <hash 1> fancy 1: hunk 1, 0 to 3
squash  <hash 2> fancy 2: hunk 2, 0 to 4
pick    <hash 3> fancy 3: hunk 1, 3 to 5
squash  <hash 4> fancy 4: hunk 1, 5 to 7
```

儲存並關閉編輯器。

如果 Git 接著要求修改 squash 後的 commit message，再確認內容、儲存並關閉即可。

第一個 squash commit 的內容等於：

```
hunk 1: 0 -> 3
hunk 2: 0 -> 4
```

第二個則是：

```
hunk 1: 3 -> 7
```

也就是原本的 3 -> 5 -> 7 被合成一次 3 -> 7。

可以確認現在 base 後面只剩兩個 commits：

```
git rev-list --count base..feat/fancy
git log --oneline --reverse base..feat/fancy
```

第一個命令應該輸出：2

如果是真實 repository，沒有事先建立 base tag，也可以先找共同祖先：

```
git merge-base v2.0 feat/fancy
```

把輸出的 commit hash 拿去做：

```
git rebase -i <hash>
```

6. Rebase，然後處理 conflict

現在把這兩個 squash commits 套到 v2.0：

```
git rebase v2.0
```

第一個 squash commit 套上去時，hunk-1.txt 會衝突。

原因是：

squash commit 想把 hunk-1 從 0 改成 3。但 v2.0 上現在已經是 5。

所以 Git 不知道這裡應該怎麼套。

hunk-2.txt 則沒有 conflict，因為 squash commit 想要的結果是 4，而 v2.0 目前也是 4。

先看狀態：

```
git status
cat hunk-1.txt
cat hunk-2.txt
```

hunk-1.txt 會看到 Git 放進去的 conflict markers，大概長這樣：

```
目前版本的起點標記 | <<<<<< HEAD
目前版本的內容      | 5
分隔線              | =====
正在 replay 的內容   | 3
feature 版本的標記   | >>>>>> fancy 1: hunk 1, 0 to 3
```

左邊那些說明文字只是為了方便閱讀，實際檔案裡只有右邊的內容。

<<<<<<、=====、>>>>>> 都是 Git 暫時加進去的標記，方便解決衝突（但我覺得這個也很抽象）。

題目規定這次 conflict 要採用 squash commit 的 new，所以最後把 hunk-1.txt 改成 3：

```
printf "3\n" > hunk-1.txt
git add hunk-1.txt
git rebase --continue
```

這裡的 git add 不只是「準備 commit」，也代表你已經告訴 Git：這個 conflict 處理好了。

如果 git rebase --continue 開啟 commit message 編輯器，確認後儲存並關閉即可。

接下來第二個 squash commit 是 3 -> 7，這次可以直接套用，rebase 應該會完成。

最後檢查：

```
git status
cat hunk-1.txt
cat hunk-2.txt
```

```
git rev-list --count v2.0..feat/fancy
git log --graph --oneline --decorate v2.0 feat/fancy
```

最後應該是：

```
hunk-1.txt = 7
hunk-2.txt = 4
```

而 v2.0 後面有兩個 feature commits。

如果 rebase 做到一半發現不想繼續，可以：`git rebase --abort`

它會把這次 rebase 放棄，回到開始之前的狀態。

至於 `git rebase --skip` 要小心。它不是「跳過這個 conflict 再繼續」，而是直接放棄目前正在 replay 的那個 commit。不了解狀況時不要隨使用。

7. 為什麼 rebase 後要 force push？

這時 remote 還留著原本那四個 commits，但本機的 feat/fancy 已經變成另外兩個 commits。

普通的 push 不會接受這種歷史改寫，所以：

```
git push origin feat/fancy
```

應該會被拒絕。

如果確認 remote 沒有人偷偷加上新的 commits，可以使用：

```
git push --force-with-lease origin feat/fancy
```

`--force-with-lease` 和單純的 `--force` 不太一樣。

它會先確認 remote branch 還是你預期的狀態。如果有人在這段時間更新過 remote，push 就會被拒絕，而不是直接把對方的 commits 蓋掉。

不過這仍然是在改寫 remote 歷史。多人共用的 branch 在這麼做之前，還是要先確認其他人沒有正在使用它。

題目模型和現實 Git 的差異

題目裡可以直接用 old、new、current 三個整數來判斷某個 hunk 有沒有 conflict。現實的 Git 沒這麼單純，它還會看檔案中的多行 context，也會處理檔案新增、刪除、重新命名，以及彼此靠得很近的修改。

這次教學把兩個 hunk 各自放在一個一行文字檔裡，所以可以把題目中的情況跑出來。